

FUNCIONES DEL SISTEMA: SELECT versión(); -- ver del SGBD database(); -- la bd en uso user();-- ver usuario connection_id();..ver id OPERADORES MATEMÁTICOS: SUMA (+) RESTA (-) MULTIPLICACIÓN (*) DIVISIÓN (/) , DIV FUNCIONES MATEMÁTICAS MOD(); ROUND(); TRUNCATE(); CEILING() o CEIL(); FLOOR(); ABS(); RAND(); PI(); POWER() o POW(); SQRT();	FUNCIONES LOGICAS CONDICIONALES: IF() CASE() IFNULL() COALESCE() FUNCIONES DE CADENA DE CARACTERES: CONCAT() GROUP_CONCAT() LEFT(), RIGTH() LENGTH() CHAR_LENGTH() TRIM(), LTRIM(), RTRIM() LOCATE() INSTR SUBSTRING(), SUBSTRING_INDEX() STRCMP TRIM() REPLACE() REVERSE() UCASE(), o UPPER(), LCASE(), o LOWER()	FUNCIONES DE FECHA Y HORA NOW(); -- fecha y hora CURDATE(); -- fecha CURTIME(); -- hora YEAR();-- año MONTH();-- mes DAY();-- día DAYOFYEAR();-- día del año MONTHNAME();-- nom mes DAYNAME(); -- nom día hour(); -- hora MINUTE(); --minuto SECOND(); -- segundo DATEDIFF (); (no es exacta) TIMESTAMPDIFF(); DATE_FORMAT(); Formula EDAD: -TIMESTAMPDIFF(intervalo (year, month, day), fecha_antigua, fecha_reciente)
Procedimientos almacenados MySQL Porción de código que se puede guardar y reutilizar. Permiten organizar mejor el código y preservar la integridad de datos. Suele suponer una mejora de rendimiento en tareas relativamente complejas. El uso de procedimientos almacenados mejorará la legibilidad del código. Borrar un procedimiento almacenado DROP PROCEDURE nombre_procedimiento;; Visualizar el código de un procedimiento SHOW CREATE PROCEDURE nombre_procedimiento; Listar procedimientos SHOW PROCEDURE STATUS WHERE DB = 'nombreBD'; Modificar un procedimiento (ciertas cosas, no todo) ALTER PROCEDURE nombreProcedimiento COMMENT 'xxx' Ventajas de los procedimientos Se reduce el tráfico de red y puede mejorar el rendimiento. Al encapsular las operaciones, nos aseguramos de que el acceso a los datos es consistente, como queremos. Es posible limitar los permisos de acceso de usuario. Evitamos problemas derivados de un mal uso de las tablas. - Mayor flexibilidad en la sintaxis SQL. Las capacidades de subsanación de errores. - Facilidad de mantenimiento. - Mayor seguridad. Accedemos a los datos controlando que se quiere mostrar y que no. Que se quiere modificar y como. Inconvenientes de los procedimientos Con las migraciones. No todos los SGBD los usan igual, por lo que se reduce la portabilidad del código. CONTROL DE FLUJO: IF, CASE y WHILE: Alteran el flujo secuencial, permitiendo la ejecución o no de parte del código, también permiten la realización de bucles.	DROP PROCEDURE IF EXISTS procedimiento1 DELIMITER // CREATE PROCEDURE nombre_procedimiento ([parámetros]) [COMMENT 'explicación de qué es lo que hace el procedimiento'] BEGIN /*Código sql*/ END// DELIMITER ; CALL procedimiento1([parámetros]); PARÁMETROS: IN: Parámetro de entrada, por defecto. OUT: Parámetros de salida. INOUT: Parámetro de entrada y salida. VARIABLES LOCALES. Asignación de valores con SET DECLARE var1, var2, var3 INT; -- declaración de variables SET var1 = 1, var2 = 2; -- asignación de variables SET var3 = var1 + var2; CONTROL DE FLUJO: IF, CASE y WHILE IF condición THEN sentencia ELSEIF condición THEN sentencia ELSE condición final END IF; CASE WHEN condición THEN sentencia WHEN condición THEN sentencia ELSE condición final END CASE; --BUCLE WHILE--(contador) WHILE condición DO --Listas de sentencias-- END WHILE;	

TRIGGERS

Un **trigger** (o **disparador**) es un objeto de la base de datos que está **asociado con una tabla** y que **se activa cuando ocurre un evento sobre la tabla**.

trigger_time: tiempo de acción del disparador.

{ BEFORE | AFTER }

trigger_event: eventos que pueden ocurrir

{ INSERT | UPDATE | DELETE }

trigger_order:

{ FOLLOWS | PRECEDES }

ELIMINACIÓN DE TRIGGER

DROP TRIGGER [IF EXISTS]

[schema_name]trigger_name

VISUALIZACIÓN DE TRIGGER

SHOW TRIGGERS;

Restricciones en Triggers

- No puedes usar la sentencia CALL
- No puedes iniciar y parar TRANSACCIONES.
- No se puede crear TRIGGERS de una tabla TEMPORAY o una VISTA.
- La creación de trigger tiene las mismas restricciones que la creación de rutinas.

Un **trigger** tiene muchas **aplicaciones** entre las que podemos destacar:

- Forzar reglas de integridad que son difíciles de definir mediante **constraints**.
- Realizar cambios en la base de datos de forma transparente al usuario.
- Sincronización entre tablas generando automáticamente valores de columnas calculadas en base a un valor proporcionado por una sentencia INSERT o UPDATE.
- Generar valores de colum automáticamente, etc..

DELIMITER //

CREATE TRIGGER [IF NOT EXISTS]

trigger_name

{ BEFORE | AFTER }

{ INSERT | UPDATE | DELETE }

ON tbl_name FOR EACH ROW

BEGIN trigger_body

END;

//

DELIMITER ;

INSERT, **NEW.col_name**

valor que se inserta en nueva fila.

OLD no es admisible

DELETE, **OLD.col_name**

valor de fila que desea eliminar

NEW no es admisible.

UPDATE **OLD.col_name** y

NEW.col_name

Valor en fila antes y después de actualizar la fila, respectivamente.

FUNCIONES DE USUARIO

Puede tener **cero o varios parámetros de entrada**, pero **sólo un parámetro de salida o retorno**.

-----DOCUMENTAR LA FUNCION -----

- QUE HACE

- PARAMETROS QUE RECIBE, RESPETANDO EL ORDEN Y EL TIPO DE DATO

- QUE DEVUELVE Y DE QUE TIPO DE DATO

REFERENCIA A VALORES DE COLUMNA **OLD** y **NEW**.

Para hacer referencia a una columna determinada, podemos preceder el nombre de la columna con un calificador **OLD** para referirse a un **valor de la fila original**, o **NEW** para referirse a un **valor en la nueva fila**.

OLD debe ser utilizado en forma de **sólo lectura**.

NEW se utiliza **para leer o cambiar los valores de columna**.

DELIMITER //

CREATE FUNCTION func_name ([parameters])

RETURNS data_type

BEGIN

-- cuerpo de la función

RETURN 1;

END//

DELIMITER ;

-- LLAMADA A UNA FUNCIÓN:

SELECT nombre_de_funcion(parámetros);

-- ELIMINAR UNA FUNCIÓN:

DROP FUNCTION IF EXISTS _nombre_funcion;

-- MOSTRAR FUNCIONES DE UNA BD:

SHOW FUNCTION STATUS WHERE DB =

nombre_base_de_datos;

Características de la función

Después de la definición del tipo de dato que devolverá la función con la palabra reservada RETURNS, tenemos que indicar las **características de la función**. Las **opciones** disponibles son las siguientes:

- **DETERMINISTIC**: Indica que la función **siempre devuelve el mismo resultado** cuando se utilizan los mismos parámetros de entrada.
- **NOT DETERMINISTIC**: Indica que la función **no siempre devuelve el mismo resultado**, aunque se utilicen los mismos parámetros de entrada. Esta es la opción que se selecciona por defecto cuando no se indica una característica de forma explícita.
- **CONTAINS SQL**: Indica que la función **contiene sentencias SQL**, pero no contiene sentencias de manipulación de datos. Algunos ejemplos de sentencias SQL que pueden aparecer en este caso son operaciones con variables (Ej: SET @x = 1) o uso de funciones de MySQL (Ej: SELECT NOW()); entre otras. Pero **en ningún caso aparecerán sentencias de escritura o lectura de datos**.
- **NO SQL**: Indica que la función **no contiene sentencias SQL**.
- **READS SQL DATA**: Indica que **la función no modifica los datos de la base de datos** y que **contiene sentencias de lectura** de datos, como la sentencia SELECT.
- **MODIFIES SQL DATA**: Indica que la función **sí modifica los datos** de la base de datos y que **contiene sentencias** como INSERT, UPDATE o DELETE.

¿COMO SE CREAN LOS NOMBRES DE VARIABLES?

o NORMAS PARA NOMINAR: PROCEDIMIENTOS , FUNCIONES y VARIABLES

-- SIEMPRE SE ESCRIBEN EN MINUSCULAS, empiezan en minúsculas

numero numDeSillasTeatro num_sillas _sillas , var3, v33333, tres

-- NO PUEDEN EMPEZAR POR UN NUMERO, PERO DENTRO DEL NOMBRE SI PUEDO PONER NUMERO

1var -no puede

var1 -si puede

-- NO PUEDEN TENER ESPACIOS EN BLANCO

-- CARACTERES ESPECIALES (SE USAN 2 (\$) y (_), PERO AQUI USAREMOS EL GUION BAJO) (_) NO ADMITE OTROS

-- UNA CONSTANTE POR CONCENSO, SE ESCRIBE EN MAYUSCULAS: EURO, PI

-- NO ESTA PERMITIDO USAR PALABRAS RESERVADAS EN EL NOMBRE DE LAS VARIABLES COMO: IF, CASE, SELECT, CREATE

-- NO SE PERMITEN: num-sillas num sillas 3var

-- POR CONCENSO NO SE UTILIZAN NI ACENTOS NI EÑES

-- PARAS NOMBRES LARGOS --superficiePoblacion, la segunda palabra en mayúscula la primere letra, o bien: superficie_poblacion, todo en minúscula separado por guion bajo

-- AUNQUE ADMITE LA Ñ Y LOS ACENTOS, POR CONCENSO NO SE PONEN,